

DSP 블록 없는 단일 비트스트림 FPGA 가속기에서의 하드웨어 인지 CNN 영상 잡음 제거

(영문 제목 안: *Hardware-Aware CNN Image Denoising on a DSP-Free, Single-Bitstream FPGA Accelerator*)

(저자 이름), (소속: 학과, 대학교), (이메일)

초안 메모 — 제출 전에 확인할 것

- [확정 전] 표시 수치: 학습이 아직 진행 중이다. 학습이 끝나면 최종 모델의 정수 PSNR로 바꾼다.
- 보드 실측 표(표 III)와 그림 2는 이전 가중치(`dn_d7_c32`)로 측정한 것이다. 최종 가중치로 보드를 다시 돌린 뒤 바꾼다. 망의 형상이 같아서 사이클 수는 같을 것으로 예상하지만, 측정으로 확인한다.
- 영문 4쪽(IEEE 2단 템플릿)에 맞추면서 분량을 조정한다. 지금은 내용을 빠짐없이 적은 상태이다.

초록

DSP 블록을 쓰지 않고 LUT만으로 곱셈기 64개를 구현한 소형 FPGA(Zynq XC7Z020) CNN 가속기에서 영상 잡음 제거를 수행한다. 이 가속기는 하나의 비트스트림이 총마다 형상만 레지스터로 바뀌 가며 신경망 전체를 실행한다. 입력은 7비트 부호 없는 값이고, 모든 출력에 ReLU가 걸린다. 이 제약에서는 잡음을 예측해 빼는 DnCNN 방식을 그대로 쓸 수 없다. 그래서 마지막 층이 깨끗한 영상을 직접 출력하고, 잡음 영상을 마지막 층에 다시 넣는 망을 설계했다. 하드웨어는 고치지 않았다. 7층 32채널 망은 가우시안 잡음 $\sigma=25$ 에서 정수(보드) 기준 BSD68 28.78 dB, Set12 29.81 dB를 얻었다[확정 전]. 같은 하드웨어 조건(클리핑, 7비트)에서 다시 측정한 BM3D보다 BSD68에서 0.41 dB 높고, 68장 중 61장에서 앞선다. 보드 출력 1,264만 개는 PC 정수 모델과 비트 단위로 일치했다. 측정 사이클은 설계 단계 사이클 모델과 1.7% 이내로 맞았다. 사이클 모델로 분석하면 이 코어는 입력 대역에 묶여 곱셈기를 42%만 쓴다. 영상을 2×2 위상으로 나눠 64채널로 처리하면 활용률이 89%가 되고, 픽셀당 사이클이 26% 줄어든다.

주제어 — 영상 잡음 제거, CNN 가속기, FPGA, INT8 양자화, 하드웨어 검증

I. 서론

영상에는 저조도 촬영, 소형 센서, 전송 과정에서 잡음이 섞인다. DnCNN [2] 같은 CNN 잡음 제거기는 고전 기법인 BM3D [1]보다 화질이 좋다. 그러나 픽셀 하나에 수만 번의 곱셈이 필요해서, GPU를 쓸 수 없는 엣지 기기에서는 전용 하드웨어가 필요하다.

FPGA CNN 가속기 연구 [6]–[8]는 주로 DSP 블록을 곱셈기로 쓴다. 그러나 소형 FPGA에서는 DSP가 가장 먼저 부족해지는 자원이고, 다른 신호처리에도 필요하다. 본 연구의 가속기는 곱셈기 64개를 모두 LUT로 구현해 DSP를 하나도 쓰지 않는다. 또 하나의 비트스트림으로 형상이 다른 모든 층을 실행한다. 이 가속기는 원래 분류용으로 설계했으며, CIFAR-10 분류 CNN을 보드에서 실행해 검증했다.

이런 정수 가속기에 잡음 제거를 올리면 두 가지 제약에 부딪힌다. 첫째, 입력 포트가 7비트 부호 없는 값이다. 둘째, 모든 출력에 ReLU가 걸려서 음수를 낼 수 없다. DnCNN은 음수가 섞인 잡음을 예측하므로 그대로 쓸 수 없다. 본 연구는 하드웨어를 고치지 않고 망을 하드웨어에 맞춘다.

본 연구의 기여는 다음과 같다.

1. **ReLU 출력 하드웨어에 맞춘 잡음 제거 망:** 마지막 층이 깨끗한 영상을 직접 출력하고, 잡음 영상을 마지막 층에 다시 입력한다.
2. **비트 단위 보드 검증:** 보드에서 7개 층 전부를 실행했고, 출력 전부가 PC 정수 모델과 일치했다. 비교 기법도 모두 같은 하드웨어 조건에서 다시 측정했다.
3. **실측으로 확인한 사이클 모델과 그 활용:** 모델로 코어의 병목(입력 대역)을 찾고, 그 병목을 피하는 망 구조(2×2 위상 분할, 64채널)를 제시한다.

II. 가속기 구조

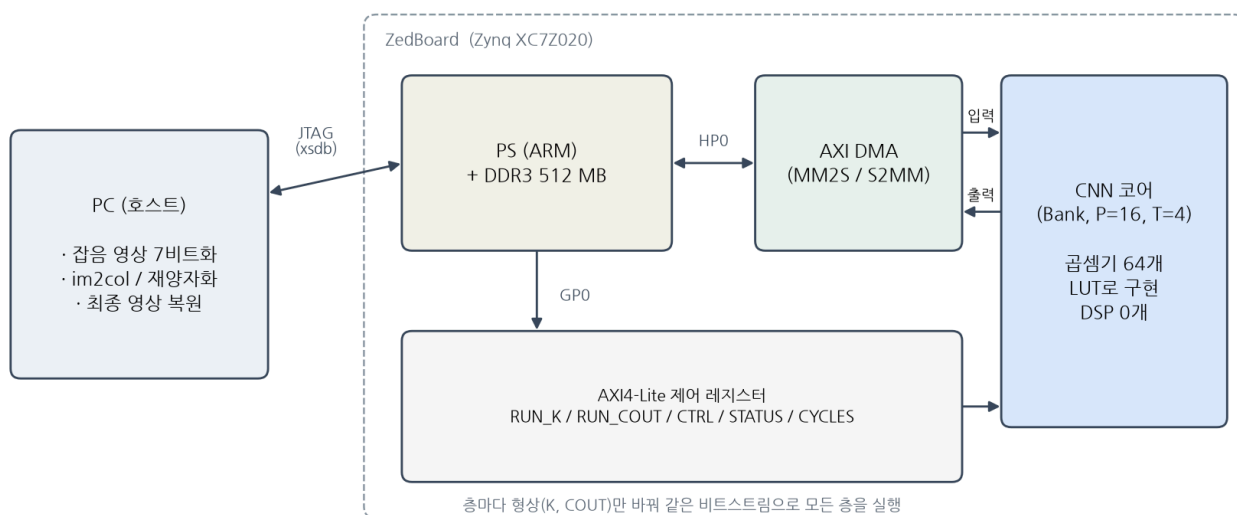


그림 1. 시스템 구성. PC가 im2col과 층 간 재양자화를 맡고, PL의 CNN 코어가 모든 합성곱을 계산한다.

연산. 코어는 창(window) 하나, 즉 활성값 K개를 받아 COUT개 채널의 $\text{ReLU}(b + \sum w \cdot x)$ 를 계산한다. 가중치는 INT8, 누산은 INT32, 활성값은 7비트이다. 곱셈기는 출력 채널 방향 16개(P)와 탭 बैंक 방향 4개(T)로, 모두 64개이다. 같은 곱셈기 수에서 출력 채널 방향으로만 늘린 구조와 비교했을 때, 이 구조가 배치·배선 후 LUT를 29%, 플립플롭을 57% 적게 썼다.

런타임 형상. 메모리는 최대 형상(K=1152, COUT=128)에 맞춰 만들었다. 층마다 RUN_K, RUN_COUT 레지스터에 실제 형상을 쓰므로, 하나의 비트스트림으로 CIFAR-10 분류망(6층)과 본 잡음 제거망(7층)을 모두 실행했다.

시스템. 데이터는 JTAG으로 DDR에 올린다. AXI DMA가 HP0 포트로 코어에 스트리밍하고 결과를 DDR에 다시 쓴다. 설계는 XC7Z020의 LUT 21%, BRAM 51%를 쓰고 DSP는 0개이다. 100 MHz 제약에서 타이밍을 만족했고(WNS +0.221 ns), 측정은 보드 기본 클럭인 50 MHz에서 했다.

사이클 모델. 입력 포트는 사이클당 활성값 하나를 받는다. 출력 채널 묶음 「COUT/P」개를 차례로 처리하고, 한 묶음은 「K/T」 사이클이 걸린다. 다음 창의 입력을 받는 동안 현재 창을 계산하므로, 창 하나의 비용은 다음과 같다.

$$C(K, \text{COUT}) = \max(K, \lceil \text{COUT}/P1 \rceil \cdot \lceil K/T1 \rceil)$$

K가 앞쪽 항보다 크면 코어는 **입력 대역에 묶인다**. 이때 곱셈기는 일부만 쓰인다.

III. 하드웨어 인지 잡음 제거 망

기본 망 (A). DnCNN [2]을 줄인 7층 $\times$ 32채널 3×3 합성곱 망이다. 배치 정규화는 합성곱에 합쳐 넣는다.

제약 1 — 7비트 입력. 잡음 영상을 $[0, 1]$ 로 자르고 7비트로 양자화한다. 학습과 평가에서 똑같이 처리한다. 실제 카메라도 센서 값을 자른다.

제약 2 — 출력에 항상 ReLU. 마지막 층이 잡음 대신 **깨끗한 영상을 직접 출력**하게 한다. 픽셀 값은 음수가 없으므로 ReLU와 충돌하지 않는다. 대신 잡음 영상을 마지막 층에 한 채널 더 입력(skip)해서, 마지막 층이 입력을 그대로 빼끼고 고칠 수 있게 한다. 마지막 층의 K는 $288 + 9 = 297$ 이 된다.

2×2 위상 분할 망 (B). FFDNet [3]처럼 영상을 2×2 위상으로 나눠 반쪽 해상도 4채널로 처리한다. 창 수가 1/4로 줄어든 만큼 채널을 64개로 늘리고 10층으로 쌓는다. 이때 $K=576$ 이고 $\lceil 64/16 \rceil \cdot \lceil 576/4 \rceil = 576 = K$ 이다. 입력 대역과 연산이 정확히 균형을 이룬다(표 II). 한 픽셀이 보는 범위는 15×15 (A)에서 약 42×42 (B)로 넓어진다.

학습. Train400 [2] 중 20장을 검증용으로 떼어 두고 380장으로 학습한다. A는 40×40 패치를 쓰고, B는 50×50 패치를 쓴다. 패치에는 뒤집기와 회전을 적용한다. 손실은 MSE, 최적화는 Adam(학습률 10^{-3} , 배치 128)이다. 학습률은 검증 PSNR이 6 epoch 동안 오르지 않으면 반으로 줄이고, 15 epoch 동안 오르지 않으면 학습을 멈춘다. 검증 PSNR이 가장 높았던 epoch의 모델을 쓴다. 테스트셋은 모델 선택에 쓰지 않았다.

양자화. 정수 추론 방식 [5]을 따른다. 가중치는 출력 채널별 대칭 INT8, 바이어스는 INT32, 활성화값은 층마다 스케일 하나를 갖는 7비트(0~127)이다. 활성화값 범위는 최대값 대신 퍼센타일로 잡는데, 퍼센타일은 테스트셋이 아닌 학습 영상 40장에서 골랐다. 층 간 재양자화는 PC에서 `clip(round(acc · s_c), 0, 127)`로 한다. skip으로 들어가는 잡음 영상은 스케일이 1/127로 다른 특징맵과 다르다. 그래서 두 스케일의 비를 영상 쪽 가중치에 미리 곱해 둔다. 하나의 누산기가 정수 곱셈만으로 두 입력을 더할 수 있다.

IV. 검증

RTL 검증. UVM [9] 검증 환경을 만들었다. 구성은 AXI4-Lite·AXI4-Stream 에이전트, 참조 모델 스코어보드, 기능 커버리지, AXI 핸드셰이크 assertion이다. UVM 1.2에서 39개, IEEE 1800.2-2020판에서 26개 테스트가 모두 통과했다. 이 환경이 실제로 버그를 잡는지 보려고 RTL에 버그 10개를 일부러 넣었고, 10개 모두 검출됐다.

보드 검증. 층마다 보드 출력을 정답(oracle)과 보드 쪽에서 출력 하나하나 비교한다. 최종 PSNR도 PC 정수 모델이 계산한 값과 같아야 통과한다.

V. 결과

실험 조건. 테스트셋은 Set12(12장)와 BSD68(68장, [4])이고, 가우시안 잡음 $\sigma=25$ 이다. 모든 기법에 **같은 잡음 영상**(고정 시드, 클리핑, 7비트)을 넣었다. 논문 수치를 옮겨 적지 않고 모두 직접 측정했다. 필터 기법의 세기는 학습 영상으로 골랐다.

표 I. 평균 PSNR (dB). 제안 기법은 정수 모델, 즉 보드가 계산하는 값이다.

방법	Set12	BSD68
잡음 영상	20.33	20.48
미디언 필터 3×3	25.43	24.89
가우시안 필터	26.56	26.13
BM3D [1]	29.95	28.37
A: 7층 × 32채널 (float / 정수)	29.82 / 29.81	28.80 / 28.78
B: 2×2 위상 분할, 10층 × 64채널 (float / 정수)	29.84 / 29.76	28.71 / 28.63

[확정 전] A, B는 학습 진행 중의 검증 최고 모델 값이다.

화질. A는 BSD68에서 BM3D보다 0.41 dB 높고, 68장 중 61장에서 앞선다. Set12에서는 0.14 dB 낮지만 12장 중 6장에서는 앞선다. Set12에서 뒤지는 차이는 대부분 Barbara(09.png, -1.82 dB)처럼 반복 무늬가 많은 영상에서 나온다. BM3D는 영상 전체에서 비슷한 조각을 모아 처리하므로 이런 영상에 강하다.

정수 변환 손실. A는 -0.01 dB(Set12)로 거의 없다. B는 -0.08 dB로 더 크다. B는 활성값 범위를 정하는 퍼센타 일에 따라 검증 PSNR이 27.2~28.5 dB로 크게 바뀌어서, 양자화에 더 민감하다.

표 II. 망별 하드웨어 비용 (256×256 영상, 50 MHz).

	A	B
파라미터	4.7만	30.1만
픽셀당 MAC	46,665	74,916
픽셀당 사이클 (모델)	1,746	1,314
픽셀당 사이클 (실측)	1,775	(측정 예정)
곱셈기 활용률	41.8%	89.1%
코어 계산 시간	2.33 s (실측)	1.72 s (모델)
한 픽셀이 보는 범위	15×15	약 42×42

병목과 위상 분할. A의 중간 층($K=288$, $COUT=32$)은 연산에 $\lceil 32/16 \rceil \cdot \lceil 288/4 \rceil = 144$ 사이클이면 되지만, 입력을 받는 데 288 사이클이 걸린다. 그래서 곱셈기는 절반 가까이 쉰다. B는 MAC이 1.6배 많은데도 사이클은 26% 적다. 곱셈기를 89% 쓰기 때문이다. 화질은 평균으로 A가 조금 높다(정수 기준 Set12 +0.05 dB, BSD68 +0.15 dB)[확정 전]. 그러나 B는 반복 무늬 영상에서 A보다 높다. Barbara(09.png)에서 28.82 대 29.04 dB, House(02.png)에서 32.08 대 32.24 dB이다. 보는 범위가 넓어진 효과로 보인다. B는 화질을 약간 내주는 대신 속도를 얻는 설계점이다.

표 III. 보드 실측 (A, Set12 05.png, 256×256, 50 MHz).

층	K	COUT	측정 사이클	사이클/창	불일치
conv1	9	32	2,097,167	32.0	0
conv2-6 (각각)	288	32	18,939,961-969	289.0	0
conv7	297	1	19,529,755	298.0	0
합계			116,326,747	픽셀당 1,775	0

보드. 7개 층의 출력 12,648,448개가 모두 PC 정수 모델과 비트 단위로 일치했다. 측정 사이클은 모델보다 1.7% 많다. 차이의 대부분은 K=9인 첫 층에서 생긴다. 창마다 고정 비용이 있어서 9 대신 32가 측정됐다. 나머지 층은 창마다 +1 사이클이다.

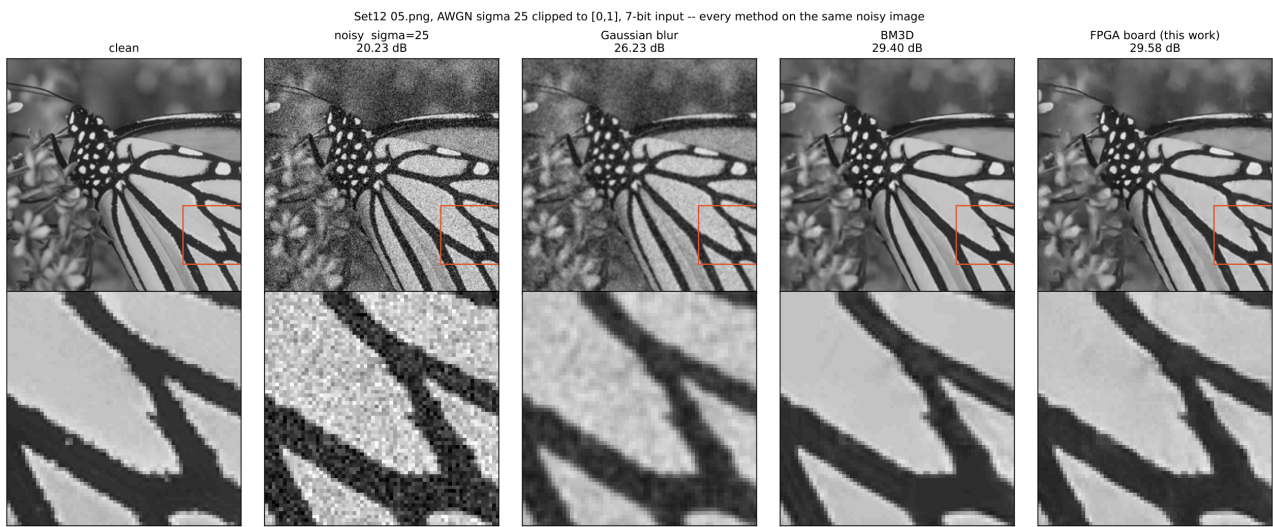


그림 2. Set12 05.png. 왼쪽부터 원본, 잡음(20.23 dB), 가우시안 필터, BM3D(29.40 dB), FPGA 보드 출력. 아래 줄은 확대 영역이다. [최종 가중치의 보드 출력으로 교체 예정]

VI. 결론

DSP를 쓰지 않는 단일 비트스트림 정수 CNN 가속기에서, 하드웨어를 고치지 않고 영상 잡음 제거를 실행했다. 7 비트 입력과 ReLU 출력이라는 제약은 깨끗한 영상 출력과 skip 입력으로 해결했다. 그 결과 같은 조건의 BM3D보다 BSD68에서 0.41 dB 높은 화질을 보드에서 비트 단위로 재현했다. 사이클 모델은 실측과 1.7% 이내로 맞았고, 이 모델로 코어가 입력 대역에 묶여 있음을 보였다. 2×2 위상 분할은 이 병목을 피해 사이클을 26% 줄인다.

한계.

- 처리 시간은 코어 계산만 센 값이다. JTAG 전송은 포함하지 않았다.
- 전력은 측정하지 않았다.
- 잡음은 $\sigma=25$ 하나만 다뤘다.

향후에는 입력 포트를 넓혀 병목 자체를 낮추고, 같은 칩의 ARM 프로세서 단독 실행과 속도를 비교할 계획이다.

참고문헌

- [1] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian, "Image denoising by sparse 3-D transform-domain collaborative filtering," *IEEE Trans. Image Process.*, vol. 16, no. 8, pp. 2080-2095, 2007.
- [2] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142-3155, 2017.
- [3] K. Zhang, W. Zuo, and L. Zhang, "FFDNet: Toward a fast and flexible solution for CNN-based image denoising," *IEEE Trans. Image Process.*, vol. 27, no. 9, pp. 4608-4622, 2018.
- [4] D. Martin, C. Fowlkes, D. Tal, and J. Malik, "A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics," in *Proc. ICCV*, 2001.
- [5] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, 2018.
- [6] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in *Proc. FPGA*, 2015.
- [7] J. Qiu et al., "Going deeper with embedded FPGA platform for convolutional neural network," in *Proc. FPGA*, 2016.
- [8] Y. Ma, Y. Cao, S. Vrudhula, and J.-s. Seo, "Optimizing loop operation and dataflow in FPGA acceleration of deep convolutional neural networks," in *Proc. FPGA*, 2017.
- [9] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2020.